

15

Upper Bound of Merge Sort $\leq O(n \log n) + O(n)$

(17)

Selection Sort

- Start by finding the smallest number.
- Swap the smallest with the first entry.
- Two Arrays - Sorted and Unsorted
- Math formula:

$$n + (n-1) + (n-2) + \dots$$

→ n بار چیک کرنے کے لیے "n" numbers check کریں گے۔ دوسری بار $(n-1)$ check کیا کریں گے۔ $(n-1)$ fix ہو گیا۔ تیسری مرتبہ $(n-2)$ (سی طرح جاری ہے)۔

- Sum of Series:

Math ← اس Series کا total نکالنے کا formula

$$\frac{n(n-1)}{2}$$

- Final Complexity:

When we open the brackets

$$\frac{n^2 - n}{2} \text{ becomes}$$

By Ignoring Constants and Non-dominant
 $\frac{1}{2}$ and n

$$\text{Answer} = O(n^2)$$

$$\therefore n^2 - \frac{1}{2}n$$

$$\boxed{n^2}$$

16

← Load, Speed, Slow, Speed, Slow

"Higher Growth rate means lower Efficiency."

Lower " " " " higher " " " " Small input ($n < n_0$)

* for small inputs, Insertion sort wins (n^2), it is faster because it has smaller constants.

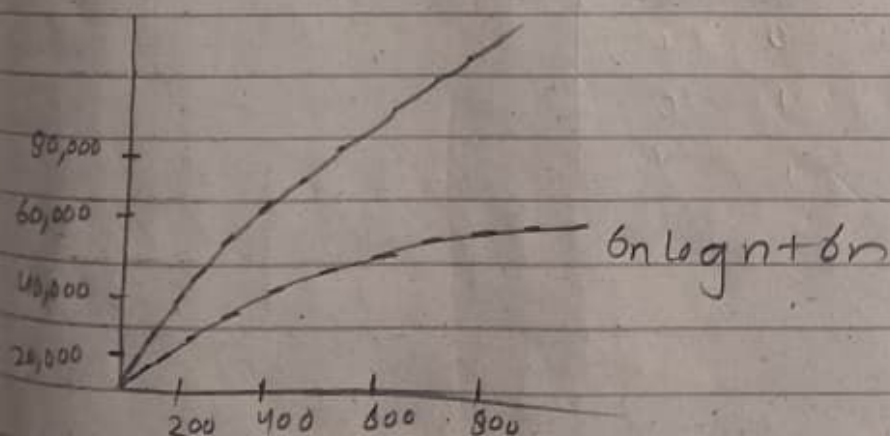
Large Inputs ($n > n_0$)

for large inputs, Merge Sort ($n \log n$) wins because quadratic growth explodes

Insertion Sort

$$T.C = O(n^2)$$

$$\text{No of ops} = \frac{1}{2}n^2$$



(Merge Sort is good for this)

Quick Sort Analysis

⇒ Algorithms

- Pick one element as Pivot
- Reorder the array so that elements $<$ Pivot comes before it, and elements $>$ Pivot go after it.
- Apply Same logic recursively to the left and right sub-tree.

Analysis of Quick Sort

E.g

[10, 80, 30, 90, 40]

(Pick) = 40

(Pivot)

[10, 30]

(Pivot = 30)

(Left side)

(Right side) [80, 90]

pivot = 90

Final

[10, 30, 40, 80, 90]

$$\Rightarrow \text{Scenario} = T(n) = T(n-1) + cn$$

List آدھی آدھی ہوتی ہے لیکن num کم ہوتا ہے

"Problem size reduce by 1 only"

$$\bullet \text{ Tree Height} = 1(n)$$

(n levels) Tree کی ہر سطح پر 1 element ہوتا ہے

Calculation =

$$\text{Series Sum} = n + (n-1) + (n-2) + \dots$$

(Same as selection sort)

When we open the brackets
 $\frac{n^2 - n}{2}$ becomes

By Ignoring ² Constants and Non-dominant
 $\frac{1}{2}$ and n

Answer = $O(n^2)$

$$\therefore n^2 - \frac{1}{2}n$$

$$\boxed{n^2}$$



→ Insertion Sort

Pick 1 element from unsorted part, compare backwards and insert at correct spot.

→ Compare 2nd with 1st

→ Compare 3rd with previous 2

→ Series $1 + 2 + 3 \dots (n-1)$

Formula same as Selection Sort

$$\frac{n(n-1)}{2}$$

$$\text{Result} = O(n^2) \approx \frac{1}{2} n^2 = \boxed{n^2}$$

→ Bubble Sort

- Compare adjacent elements pairs
- Largest element moves to last $(n-1)$ checks
- Second largest moves to 2nd last $(n-2)$ check
- Counting decreases i.e. $(n-1), (n-2) \dots$

$$\text{Formula} = \frac{n(n-1)}{2}$$

$$\text{Time Complexity} = O(n^2)$$

→ Selection, Insertion and Bubble Sort have same formula i.e.

$$\frac{n(n-1)}{2}$$

2

⇒ Efficiency:

"Zameen Asman Ka Difference"

500 pages linear search میں Book کی page "1000" ہے
جس کی BS صرف 10 pages میں ڈھونڈ لے گا

$$2^{10} \approx 1000 \Rightarrow O(\log_2 n)$$

Binary Search

$O(\log_2 n)$

Binary Search کی پہلی شرط یہ ہے کہ List پہلے سے Sort ہوگی

⇒ Strategy:

"Divide and Conquer"

ہم Step میں List کو دو حصوں میں توڑ دیا جاتا ہے اور آدھا
حصہ پھینک دیا جاتا ہے۔

⇒ Math Formula:

$$O(\log_2 n)$$

"2" Base اس لیے کہیں کہ ہر بار ہم "2" سے divide کرتے
ہیں۔ اگر 1000 items ہیں تو "10" Steps

⇒ Complexity = $O(\log_2 n)$

"Logarithmic base represents having
the input size repeatedly."

(From Slides)

Mids preparations

Dr. Saeed Arif

Searching Algorithms

(Linear and Binary Search)

⇒ Linear Search : Phone Directory
(Unsorted) (Scanning)

Phone numbers "directory" ← اگر آپ کو کسی کا نام دیا ہوگا تو پھر
First page ← اس کا sort ہے تو آپ کو پھر
Last ← سب numbers کو check کرنا پڑے گا۔

You have to scan the entire list because
"No order"

$O(n) \Rightarrow$ slow.

⇒ Binary Search : English Dictionary
(Sorted) (Divide & Conquer)

← اگر آپ کو "Sort" کا مطلب دے دیتا ہے dictionary کو آپ
First page میں open کرتے ہیں اور Middle میں دیکھتے ہیں اور
اگر "S" ہے تو پھر آدھا Part دے دیتے ہیں۔

"Open Middle". If target is greater,
discard left half. Repeat!

$O(\log n) \Rightarrow$ fast

⇒ Values:

Value 39, Index 5
Ignore Index "0-5"

Value at list[5] is 39. Splits list into
Left (0-4) and Right (6-11)

(Slide 06)

Iterative Algorithm

<pre>{ int mid; int first = 0; last = list.length() - 1; while (first <= last)</pre>	Initialization Setting Boundaries Loop
<pre> mid = (first + last) / 2; if (list[mid] == target); if (target < list[mid]); else first = mid + 1;</pre>	Calculating Pivot Success Case Left half Right half

Variables بنائے، first شروع میں 0 ہے

Last آخری Index ہے (size-1)

جب تک first, last سے 2 سے نہیں نکل جاتا، ڈھونڈتے

رہتے۔ اگر Cross کر لیتے مطلب number نہیں ملا۔

Calculate Mid point کیا۔

اگر Mid مل گیا تو وہی Return کردو (Game over)

اگر target چھوٹا ہے تو last کو اٹھا کر mid-1

پر لے آؤ۔ Right side کٹ گئی۔ (Peeche jao)

Slide (4)

⇒ Assumption:

- Items in the array must be sorted
- Algorithm begins at Middle Always

⇒ The Decision: Comparison Logic

∴ Target < Middle (1) (2)
second half - target
→ "Left side"

∴ Target > Middle (1) (2)
"Right side"

(Slide 5)

Visualization

→ Indices:

"12" Total Elements (0 to 11) Indices ← Array ← elements

[0] [1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11]
List = [4, 8, 19, 25, 34, 39, 45, 48, 66, 75, 89, 95]

(Mid)

→ Finding Mid =

$$\text{Mid} = (\text{Left} + \text{Right})$$

$$= \frac{0 + 11}{2} = 5 \text{ (Integer Division)}$$

∴ "39" is at Index 5 - "0 + 11 = 5" is Start
→ Mid is

(Slide 08)

Tracing Example

Iteration	First	Last	Mid	List[Mid]	Comparison
1	0	11	5	39	Logic Target (89) > 39 right side $\rightarrow$ First = $\frac{0+11}{2}$ $\Rightarrow$ (mid + 1) now 8 $\leq$ 11 $\rightarrow$ (8)
2	6	11	8	66	New mid $(6+11)/2 = 9$ Value = "66" $89 > 66$ Right side again $8+1 = 9$ New first = 9
3	9	11			New Mid = $(9+11)/2 = 10$ Value = "89" $89 == 89$ "found!" True

← اگر ہمارا target first سے mid+1 کے درمیان ہے،
(Age Jao)۔ اسے cut Left side

Slide (7)

(Recursive Algorithm)

(1) find(list, target, first, last) ⇒ Parameters
" اس function کو پہلے بار first اور
last دیتے ہیں - "

(2) If (first > last) return -1 ⇒ Base Case
" اگر range end پر پہنچ گئی اور
number نہیں ملا تو -1 return کریں "

(3) return find(--mid+1, last) ⇒ Recursion (01)

" اگر target پہنچا، تو function دوبارہ
call کرو لیکن first سے mid+1 تک "

(4) return find(---, first, mid-1) ⇒ Recursion (02)

" اگر target پہنچا، تو
--- side پر call کرو۔ "

Slide 10

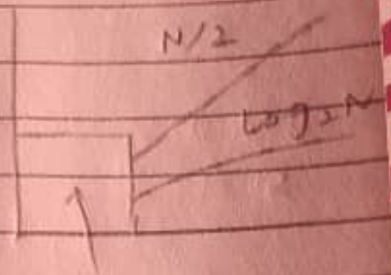
Trade off & Graph

Comparing graphs of Linear vs Binary

⇒ Linear Growth: ($N/2$)

Performance degrades

Linearly with input size



Time کی شرح سے، Increase data کے مطلب ہے
- Costly ہے -

⇒ Binary Growth ($\log_2 N$):

→ Also called Logarithmic

Growth

→ Extremely Efficient for large Input

double Data کے مطلب ہے، بھی وقت کم رہے گا -

⇒ Trade off:

Benefit: Very fast

Disadvantage: Array must be sorted.

(0) (1) (2) (3) (4) (5) (6) (7) (8) (9)
 [ant, cat, chicken, cow, deer, dog, fish, goat, horse, camel, snake]
 (13)

(Slide 09)

String

Example:

Search Item	Trace Logio	Result
-------------	-------------	--------

Cat

1- Mid Dog ha

Cat < Dog → Left

Returns

2- Mid chicken ha

True

Cat < Chicken → Left

3- Mid Cat ha

"Found True"

Zebra

1- Zebra > Dog → Right

2- Zebra > Horse → Right

3- Zebra > Snake → Right

Returns

(Array
End)

False

Fish

1- Fish > Dog → Right

2- Fish < Horse → Left

3- Mid Fish ha

"Found True"

Returns

True

(Slides 13)

Search Tracing in BST

- | | Logic | Action |
|----------|-------------------------------------|---------------------------------|
| 1) Start | Root 42 ha, We need 50
$50 > 42$ | Right branch
par jaein
gy |
| 2) Next | Next node is 47
$50 > 47$ | Again move
to right |
| 3) Next | Next node is 67
$50 < 67$ | Move to
Left |
| 4) End | 67 k left pr
nothing | 50 Not found |

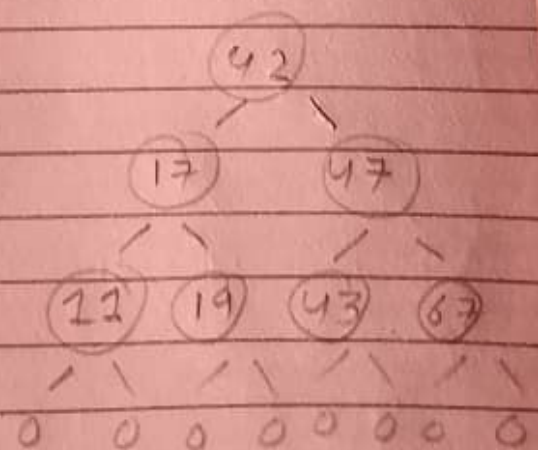
⇒ Comparisons

3 nodes, 3 comparisons ←
we check

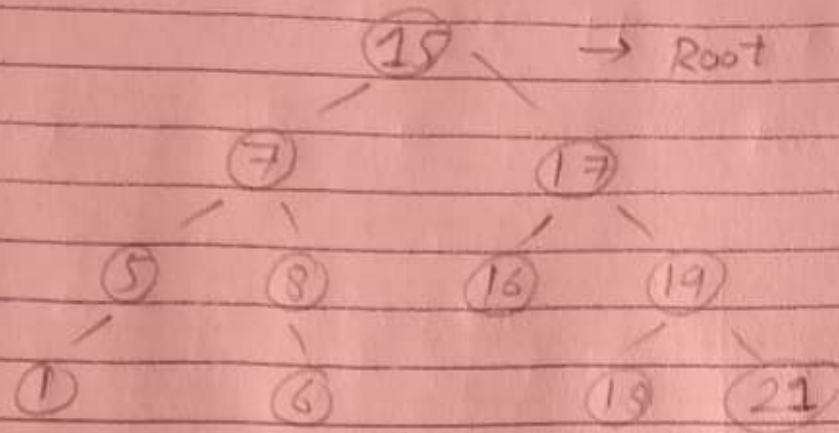
(42, 47, 67)

$$3 = \log_2 8$$

⇒ (Total 3 Comparisons performed)



Binary Search Tree



⇒ Root :-

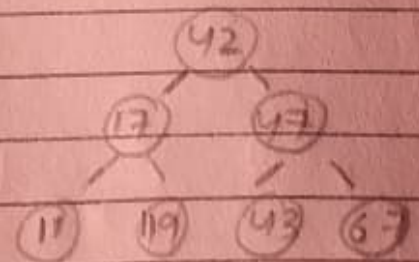
Top most node of tree.

⇒ Rule: BST property

"for any node, all left descendants are smaller, and all right descendants are larger".

⇒ Full Balanced:

1. All Equal Sides of a Binary Search Tree is Balanced
- It is end Benefit is



The tree has roughly equal height on left and right, ensuring

$O(\log n)$.

Master Method (01)

Larger Integer Multiplication Problem =

Dr. Saeed Arif نے ان Notes میں discuss کیا ہے کہ یہ
Computer 3 numbers multiply کرتا ہے

→ The Concept:

Divide and Conquer

اگر ہمارے پاس دو بڑے نمبر ہوں (x, y) تو انہیں ہم direct
multiply نہیں کرتے بلکہ انہیں دو حصوں میں divide کر دیتے ہیں

→ Breaking

$$x = 10^{n/2} a + b$$

$$1200 + 34 = \boxed{1200}$$

$$y = 10^{n/2} c + d$$

$$5600 + 78 = \boxed{5678}$$

اگر $x = 1234$ ہے تو ہم $12(a)$ اور $34(b)$ میں توڑ
دیتے ہیں۔ basically shifting -

→ Standard formula:

$$x \cdot y = ac \cdot 10^n + (ad + bc) 10^{n/2} + bd$$

ہے جب ان کو بڑے حصوں کو multiply کرتے تو اوپر دی گئی Equation
میں جاتی ہے۔

→ The problem:

"4 Recursive Calls"

(i) $a \times c$ (ii) $a \times d$ (iii) $b \times c$ (iv) $b \times d$

(Slide 14)

Summary

⇒ Conditions:

- $\Rightarrow$ BSTs, sorted array, Binary Search $\leftarrow$

⇒ Power:

- $\Rightarrow$ (cut in half) Step $\frac{1}{2}$ $\leftarrow$

⇒ Final Complexity:

$$O(\log N)$$

Question: How many times we cut in half?

$\Rightarrow$ جب تک صرف "1" Element نہ رہے (b. r) $\Rightarrow$ $\log N$ $\leftarrow$

(03)

← اب ہمیں صرف 3 درجہ Calculation کرنی پڑے گی۔ Computer
بہت آسان۔

→ New Calls

$$P_1 = ac \quad (\text{left side})$$

$$P_2 = bd \quad (\text{right side})$$

$$P_3 = (a+b)(c+d) \quad (\text{Trick/Hide Part})$$

→ New Recurrence:

$$T(n) = 3T(n/2) + O(n)$$

← اب "a=3" سے اس complexity $O(n^{1.58})$ ہو جائے گی، $O(n^2)$ سے بہتر ہے۔

→ So Master Method is a technique to find the speed of any divide and conquer algorithm in terms of Big O.

$$\text{Formula: } T(n) = aT(n/b) + O(n^d)$$

Where

n = Input

a = No of sub-problems / recursive calls

b = factor by which sub-problem reduces

d = Exponent to input work done outside req. call.

① Problem size کتنا، کتنے حصوں میں توڑا گیا (Merge = 2)

② Problem size کتنا، کتنے حصوں میں توڑا گیا (Binary = 1)

③ Extra کتنا تک رہا؟ Divide & Conquer

Time
Constant = d, linear d = 1

Recurrence

$$T(n) = 4T(n/2) + O(n)$$

← کیوں 4 multiplications ہوتی ہیں اس لئے $a=4$ اور
 ← slow ہے $O(n^2)$ complexity جاتی ہے

Part (2)

Guess Trick

⇒ Target:

Reduce Calculations from 4 to 3.

← ہمیں کسی طرح ac اور bd کو الگ الگ calculate کرنے سے روکنا ہے

→ Trick

$$(a+b)(c+d) = ac + ad + bc + bd$$

← اگر ہم $(a+b)(c+d)$ 3 multiply کرتے ہیں اس result میں
 ← $hide (ac + bd)$ req. part

→ Extraction

$$(ac + bd) = (a+b)(c+d) - ac - bd$$

← اگر ہم Total Product میں سے ac اور bd کو نکال دیں
 تو beech مل جاتا۔

$$= ac + ad + bc + bd - ac - bd$$

$$= ad + bc$$

(05)

Example 02.

Binary Search

$$T(n) = 1T(n/2) + O(n^d) / C(\text{constant})$$

$$a = 1, b = 2, d = 0$$

→ Calculation

$$b^d = 2^0 = 1$$

→ Comparison

$$a = b^d = 1 = 1 (\text{True})$$

→ Complexity

$$O(n^0 \log n) = 1 \cdot \log n = O(\log n)$$

⇒ The 3 Cases of Master Method

(Case)	Comparisons	Conditions	Result	E-g
Case-I	$a = b^d$	اگر sub-probs کی Quantity speed کے برابر Size equal ہو	$O(n^d \log n)$	Merge Sort Evenly distribute work
Case-II	$a < b^d$	اگر sub-probs کی Quantity tree کا عمل میں سے بہتر ہو	$O(n^d)$	Geometric Algorithms
Case III	$a > b^d$	اگر sub-probs کی Quantity tree کا عمل fast سے بہتر ہو	$O(n^{\log_b a})$	Quasi Trick $3 > 2$, so $O(n^{\log_2 3})$

Example: 01

Merge Sort

$$\text{As } T(n) = 2T(n/2) + O(n)$$

$$a = 2, b = 2, d = 1$$

→ Calculation

$$\text{Check } b^d = 2^1 = 2$$

→ Comparison

$$a \text{ vs } b^d = 2 = 2 \text{ (Equal)}$$

→ Result

$$a = b^d \text{ (Case 1)}$$

→ Formula

$$O(n \log n)$$

→ Simplified Work	$c \cdot n^d \cdot (a/b)^j$	← اور دانی Equation کی Simple کیا - اس میں "c" اور n^d constants باہر نکال دیے
→ Total Work Formula	$\sum_{j=0}^{\log n} c \cdot n^d \cdot (a/b)^j$	← ہم نے سب Levels کا sum کر لیا اب ساری Game (n) "sum" summation کی ہے -

Part 2: The Battle - RSP vs RWS

"Forces of Good vs Evil"

Term	Full form	Meaning
RSP	Rate of Subproblem Poliferation	"Force of Evil" ← یہ وہ speed ہے جس سے problems بڑھ رہی ہیں (multiply) یہ "a" پر depend کرتا ہے -
RWS	Rate of Work Shrinkage	"Forces of Good" ← یہ وہ speed ہے جس سے کام کا size کم ہو رہا ہے - یہ "b" پر depend کرتا ہے
Compare	Ratio $\frac{a}{b^d}$	← سارا result اس بات پر depend کرتا ہے کہ Evil (a) زیادہ powerful ہے یا Good (b ^d) -

(03)

Post 03:- Solving 3 Cases

Cases $\frac{a}{b} \geq 1$ or Solve formula & Summation & Now
- Master Method 3. 3

Case-I : Balance State

Step	Logic	Explanation
Condition	$a = b^d$	size, speed ki problem - equal speed ki
Ratio	$r = \frac{a}{b^d} = 1$	kyon ki upar se down same - Ans 3
Summation	$\sum_{j=0}^k 1 = k + 1$	root 1, level k - $\log_b n$ k - $k = \log_b n$
Calculation	$\sum_{j=0}^k 1 = k + 1, \text{ now}$ $c \cdot nd(\log_b n + 1)$	Formula mein put kia
Big O Result	$O(nd \log n)$	constant c and 1 ignore
Concept	Work is equal	Its means on every level of tree equal work is done.

Master Method Proof

Part (1)

Recursive Tree Setup

↳ tree Start, Proof Recursive Tree
 - Level. Check

Component	Math Symbols	Explanation
→ Recurrence relation	$T(n) = aT(n/b) + O(n^d)$	Math Equation to solve
→ Tree Levels	$j = 0, 1, 2, \dots, \log_b n$	a is root and $\log_b n$ leaves (last level)
→ Sub problem Count	a^j	problems per level $j \leftarrow$ i.e. a^0 at level 0, a^1 at level 1, a^2 at level 2, etc.
→ Size of Problem	n/b^j	problem per level $j \leftarrow$ size of problem at level j is n/b^j
→ Work at level j	$a^j \cdot c \cdot (n/b^j)^d$	Important step $\leftarrow$ (Number of nodes) $\times$ (Work per node)

(5)

Case III - Leaves Dominates

جی جب "Evil force" زیادہ ہو۔ یعنی problems تیزی سے Increase ہوتی ہیں۔

ہوں۔

Step Condition	Math Logic $a > b^d$	Explanation Problem کے بڑھنے کی speed, size کم ہوتے گا سے زیادہ ہے۔
Series	$1+2+3+\dots+$ Last term	Series بڑھتی جاتی ہے اس میں سب سے بڑا Last number والا ہوگا۔
Last Term	$a^{\log b n}$	Tree کے سب سے نیچے (Leaves) پر nodes کی strength
Math Trick	$a^{\log b n} = n^{\log b a}$	ہم n اور a کو swap کر رہے ہیں logarithmic, using math property
Calculation	$c \cdot nd \left(\frac{n^{\log b a}}{nd} \right)$	Algebra کے بعد cut $c \cdot nd$ کرتا
Big - O Result	$O(n^{\log b a})$	Ignoring constants
Concept	Most work at leaves	مسارا کا کم Tree کے نیچے والا جو حصہ ہوتا (Leaves) پر ہوتا ہے۔

(06)

→ Summary for Exam

→ I prove 3 Master's method in 10* Paper 1/15

(1) Tree create krein

- $\sum_{i=1}^n \log_i$ and $(\frac{a}{b})^j$ work to level j & G

(2) Series bnsein

Geometric Series $\sum \left(\frac{a}{b}\right)^j$

(3) Case-I ($= 1$)

Sum $\log n$ bn jata $\rightarrow O(n \log n)$

(4) Case-II (< 1)

Sum Constant bn jata $\rightarrow O(n)$

(5) Case-III (> 1)

Sum mein last term heavy ho jati
 $\rightarrow O(n^{\log_b a})$

(04)

Case II- Root dominates

• جب Good forces زیادہ ہوں (یعنی $a < b^d$) problem تیزی سے چھوٹی ہو رہی ہو تو ---

Step Condition	Math logic $a < b^d$	Explanation Subproblem میں size تیزی سے Shrink ہو رہا ہے۔
Ratio (r)	$r < 1$ (e.g. 0.5)	Ratio < 1 چھوٹا ہے۔ Geometric Series ($1 + 0.5 + 0.25 + \dots$)
Summation Rule	Converges to Constant	When series is less than 1, then sum is a very small constant number (Not go to infinity).
Calculation	$c \cdot nd$ (Constant)	
Big O Notation	$O(nd)$	Ignoring c and Constants
Concept	Most work at root	• سب سے زیادہ ج.د. root پر ہوتا ہے، • سب سے کم ج.د. کم سے کم ہوتا ہے۔